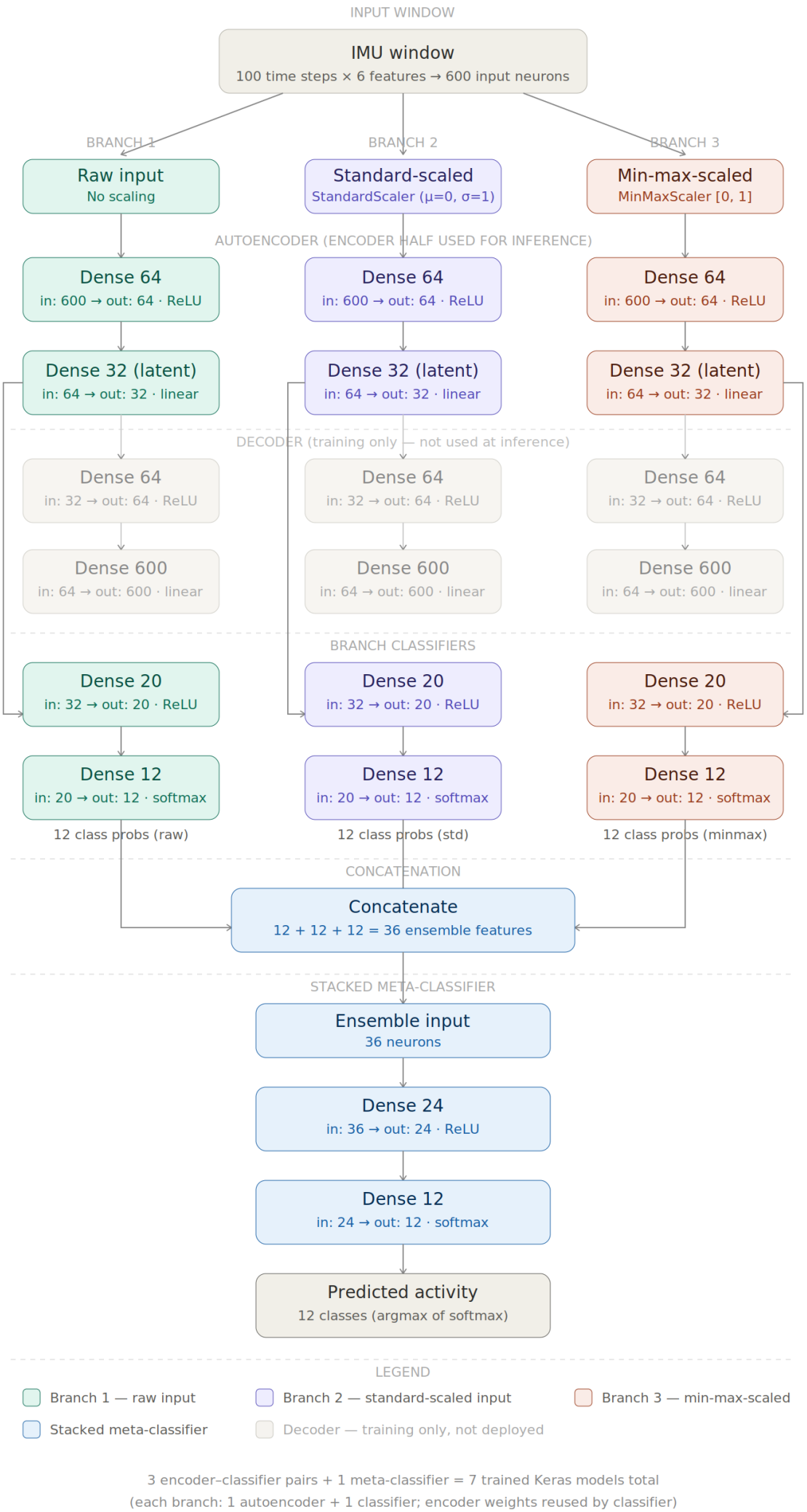
Part I

https://studio.edgeimpulse.com/public/1013920/live

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 63 ms, inference 544 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  Idle: 0.996094
  Lift: 0.000000
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 63 ms, inference 542 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  Idle: 0.000000
  Lift: 0.996094
```

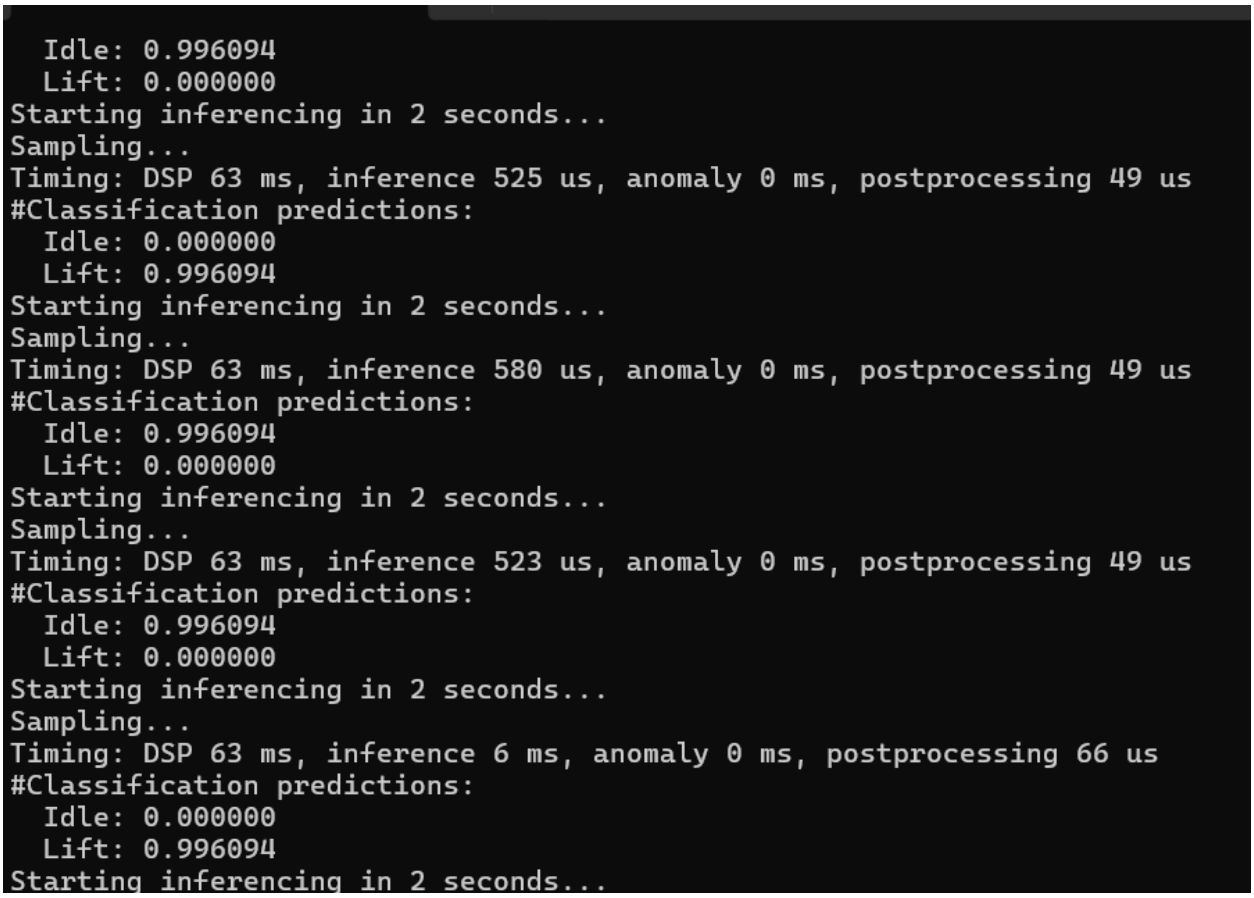
Part II

1)


INPUT WINDOW
IMU window
100 time steps × 6 features → 600 input neurons
BRANCH 1
BRANCH 2
BRANCH 3
Raw input
No scaling
Standard-scaled
StandardScaler (μ=0, σ=1)
Min-max-scaled
MinMaxScaler [0, 1]
AUTOENCODER (ENCODER HALF USED FOR INFERENCE)
Dense 64
in: 600 → out: 64 · ReLU
Dense 64
in: 600 → out: 64 · ReLU
Dense 64
in: 600 → out: 64 · ReLU
Dense 32 (latent)
in: 64 → out: 32 · linear
Dense 32 (latent)
in: 64 → out: 32 · linear
Dense 32 (latent)
in: 64 → out: 32 · linear
DECODER (training only — not used at inference)
Dense 64
in: 32 → out: 64 · ReLU
Dense 64
in: 32 → out: 64 · ReLU
Dense 64
in: 32 → out: 64 · ReLU
Dense 600
in: 64 → out: 600 · linear
Dense 600
in: 64 → out: 600 · linear
Dense 600
in: 64 → out: 600 · linear
BRANCH CLASSIFIERS
Dense 20
in: 32 → out: 20 · ReLU
Dense 20
in: 32 → out: 20 · ReLU
Dense 20
in: 32 → out: 20 · ReLU
Dense 12
in: 20 → out: 12 · softmax
Dense 12
in: 20 → out: 12 · softmax
Dense 12
in: 20 → out: 12 · softmax
12 class probs (raw)
12 class probs (std)
12 class probs (minmax)
CONCATENATION
Concatenate
12 + 12 + 12 = 36 ensemble features
STACKED META-CLASSIFIER
Ensemble input
36 neurons
Dense 24
in: 36 → out: 24 · ReLU
Dense 12
in: 24 → out: 12 · softmax
Predicted activity
12 classes (argmax of softmax)
LEGEND
Branch 1 — raw input
Branch 2 — standard-scaled input
Branch 3 — min-max-scaled
Stacked meta-classifier
Decoder — training only, not deployed
3 encoder–classifier pairs + 1 meta-classifier = 7 trained Keras models total
(each branch: 1 autoencoder + 1 classifier; encoder weights reused by classifier)

2) The notebook compresses the models using a pruning and quantization-aware training pipeline. First, pruning is applied with a polynomial decay schedule. The sparsity starts at **10%** and increases to **80%**, beginning at step 0 and ending at step 500. This gradually forces many weights toward zero while the model is still being trained. After pruning, the pruning wrappers are stripped from the model. This is important because the wrappers are only needed during pruning training, not for final deployment. Stripping them creates a regular Keras model with sparse weights, which is easier to pass into the next compression step. After that, quantization-aware training is applied. QAT simulates the numerical effects of quantization during training, so the model can adjust to the lower precision it will have when converted to an int8 TensorFlow Lite model. The notebook also uses a mask-enforcement step during and after QAT. This matters because regular QAT updates could accidentally make previously pruned weights nonzero again. The mask-enforcement callback reapplies the pruning masks so the weights that were pruned remain zero. This preserves the sparsity gained from pruning while still allowing the model to adapt to quantization. Finally, the model is converted to a fully int8 TensorFlow Lite model. A representative dataset is used during conversion so TensorFlow Lite can estimate proper scale and zero-point values for the int8 tensors. This is important because poor quantization ranges can reduce accuracy. Pruning helps by reducing the number of meaningful weights, while int8 quantization reduces the memory needed to store weights and activations. Together, these methods make the model more practical for a resource-constrained board like the Arduino Nano 33 BLE Sense.

3)



For the resting board condition, the deployed model output was mostly acceptable. When the Arduino Nano 33 BLE Sense was placed down and kept still, the predictions generally behaved as expected and showed that the model was running properly on the board. Overall, the output worked well enough for the resting test condition. One issue I noticed was that the first reading incorrectly classified the motion as a lifting activity, even though the board was laying still. This could have been caused by small sensor noise, the board settling after being placed down, or a mismatch between the orientation of the real board and the orientation used in the training data. After that first reading, the output appeared more reasonable, so the issue seemed to be temporary rather than a complete failure of the model. To improve deployment reliability, one practical change would be to ignore the first prediction after startup or after moving the board, since the first sensor window may contain unstable data. Another improvement would be to average or majority-vote across several prediction windows instead of trusting a single prediction. This would make the output more stable and reduce the chance of one noisy reading causing an incorrect classification.

4) After deploying the ensemble model on the Arduino Nano 33 BLE Sense, I did observe some unexpected prediction behavior. In some cases, the model gave a prediction that did not match the actual board motion, such as thinking the board was doing a lifting-type motion even when it was resting or being moved differently. However, the model did not completely fail, since most of the output still appeared reasonable once the board was stable. This behavior can happen because the model was trained using the original dataset, while the live Arduino sensor data may not perfectly match that dataset. The original training data may have been collected with the sensor placed on a different part of the body, at a different orientation, or with slightly different motion patterns. The Arduino board may also have a different axis orientation, meaning that acceleration or gyroscope readings along X, Y, and Z may not line up the same way as in the training data. Small differences in unit conversion, feature scaling, sampling rate, or window formatting can also affect the model's predictions. Another issue is that real-time testing is less controlled than the training data. If the board is picked up, placed down, tilted, or moved between tests, the first few windows may contain mixed or noisy motion data. Because of this, the model may briefly predict an unexpected class or repeatedly favor one class if the live sensor pattern looks closer to that class than the intended motion. Overall, the deployment results are usable, but they show that matching the Arduino preprocessing, sensor orientation, and testing motion to the original training setup is very important for reliable predictions.